
How the Program Works — Complete Flow

Data Storage

This system does NOT use a database. All data is stored in plain text files located in `src/main/resources/data/` folder. Each record is one line in the file and fields are separated by commas.

```
admins.txt
students.txt
lecturers.txt
courses.txt
courseCategories.txt
registrations.txt
payments.txt
feedbacks.txt
schedules.txt
rooms.txt
courseMaterials.txt
```

Uploaded files like payment receipts and course materials are stored in:

```
uploads/payments/
uploads/materials/
```

How Login Works

1. User opens browser and types the URL
 2. `@GetMapping("/login")` runs and shows login page
 3. User types email and password and clicks submit
 4. `@PostMapping("/login")` in `AdminController` receives email and password
 5. System checks Admin first by calling `adminService.login()`
 6. `AdminService` calls `adminRepository.findAll()` which reads `admins.txt` line by line
 7. Each line is converted to an Admin object
 8. Service loops through and checks if email and password match
 9. If match found — session is set with `userRole=ADMIN` and redirected to dashboard
 10. If no match — same process repeated for Student by reading `students.txt`
 11. If no match — same process repeated for Lecturer by reading `lecturers.txt`
 12. If nobody matches — error message shown on login page
-

How Dashboard Works

1. After login `@GetMapping("/dashboard")` runs

2. Controller reads `userRole` from session
 3. If ADMIN — calls multiple services to get total counts of students, lecturers, courses, registrations, pending payments, feedbacks and adds them to Model
 4. If STUDENT — gets that student's registrations and payments only
 5. If LECTURER — gets that lecturer's schedules only
 6. Model carries all this data to `dashboard.html` for display
-

How Admin Management Works

Adding Admin:

1. Admin fills add form and submits
2. `@PostMapping("/admins/add")` receives form data via `@RequestParam`
3. Controller generates ID using `"ADM" + System.currentTimeMillis()`
4. Calls `adminService.addAdmin()`
5. Service checks if ID already exists and if email already used
6. If all valid — calls `adminRepository.save()`
7. Repository opens `admins.txt` in append mode and writes new line

Editing Admin:

1. Admin submits edit form
2. Controller gets existing admin to retrieve current password
3. If new password typed use new one otherwise keep old
4. Calls `adminService.updateAdmin()`
5. Service calls `adminRepository.update()`
6. Repository reads entire `admins.txt`, rewrites every line, replaces only the matching admin line

Deleting Admin:

1. `@GetMapping("/admins/delete/{adminId}")` receives `adminId` from URL via `@PathVariable`
2. Calls `adminService.deleteAdmin()`
3. Service checks if this is the last admin — if yes stops deletion
4. If not last admin — calls `adminRepository.delete()`
5. Repository reads all admins, rewrites file excluding deleted admin line

Searching Admin:

1. `@GetMapping("/admins/search")` receives search text via `@RequestParam`
 2. Calls `adminService.searchByName()`
 3. Service loops through all admins checking if name contains search text
 4. Returns matching list to controller
 5. Controller puts results in Model and shows on page
-

How Student Management Works

Adding Student:

1. Admin fills student form and submits
2. `@PostMapping("/students/add")` receives data
3. Controller calls `studentService.generateStudentId()` which reads `students.txt` finds highest ID number and returns next one like S001 S002
4. Creates Student object and calls `studentService.addStudent()`
5. Service checks if ID and email already exist
6. If valid — calls `studentRepository.save()` which writes new line to `students.txt`

Editing Student:

1. Admin submits edit form with `studentId` in URL
2. Controller gets existing student to preserve password if not changed
3. Calls `studentService.updateStudent()`
4. Repository rewrites `students.txt` replacing only that student line

Deleting Student:

1. Controller receives `studentId` from URL
 2. Calls `studentService.deleteStudent()`
 3. Repository rewrites `students.txt` excluding that student line
-

How Lecturer Management Works

Same flow as Student Management but reads and writes to `lecturers.txt`. Lecturer has extra fields `department` and `specialization`. Lecturer ID is generated same way as Student using L001 L002 format.

How Course Management Works

Adding Course:

1. Admin fills course form including selecting a category
2. Controller generates `courseId` using `"CRS" + System.currentTimeMillis()`
3. Gets `CourseCategory` object using `categoryId`
4. Creates Course object with category inside it
5. Calls `courseService.addCourse()`
6. Service checks if course name already exists
7. Repository writes new line to `courses.txt` storing `categoryId` not the full category object

Reading Course:

1. `courseRepository.findAll()` reads `courses.txt` line by line
 2. For each line it calls `categoryRepository.findById()` to get the full category object
 3. Creates complete `Course` object with category attached
 4. Returns full list to service then to controller
-

How Course Category Management Works

Simple CRUD. Categories stored in `courseCategories.txt` as two fields — `categoryId` and `categoryName`. When a course is saved only the `categoryId` is stored. When course is loaded the full category is fetched from `courseCategories.txt` using that ID.

How Registration Works

Student Registering:

1. Student clicks register on a course
2. `@GetMapping("/registrations/register")` gets `courseId` from URL
3. Gets `studentId` from session
4. Calls `registrationService.addRegistration()`
5. Service checks if student already registered for this course
6. Service checks if course is full using `course.isAvailable()`
7. If valid — creates `Registration` with status `PENDING` and saves to `registrations.txt`

Admin Approving:

1. Admin clicks approve on a registration
2. `@GetMapping("/registrations/approve/{registrationId}")` runs
3. Calls `registrationService.approveRegistration()`
4. Service finds registration checks status is `PENDING`
5. Calls `registration.approve()` which changes status to `APPROVED` and increases `course.enrolledStudents` count
6. Repository updates `registrations.txt` with new status
7. Repository also updates `courses.txt` with new enrolled count

Admin Rejecting: Same as approve but calls `registration.reject()` which changes status to `REJECTED`.

How Payment Works

Student Submitting Payment:

1. Student fills payment form with amount, method and uploads PDF receipt

2. `@PostMapping("/payments/add")` receives data and file as `MultipartFile`
3. Calls `paymentService.addPayment()`
4. Service checks file is not empty
5. Service checks file is PDF only
6. Service checks amount is greater than zero
7. Calls `paymentRepository.saveUploadedFile()` which saves PDF to `uploads/payments/` folder with timestamp prefix
8. Creates Payment object with status PENDING
9. Repository saves record to `payments.txt` storing the saved filename

Admin Approving Payment:

1. Admin clicks approve
2. Calls `paymentService.approvePayment()`
3. Service finds payment checks status is PENDING
4. Calls `payment.processPayment()` which changes status to COMPLETED
5. Repository updates `payments.txt` with new status

Admin Cancelling Payment: Same as approve but calls `payment.cancelPayment()` which changes status to FAILED.

Viewing Receipt:

1. `@GetMapping("/payments/receipt/{paymentId}")` runs
2. Gets payment finds receipt filename
3. Builds file path to `uploads/payments/` folder
4. Returns file as `ResponseEntity` with content type PDF so browser opens it directly

Deleting Payment:

1. Service finds payment
2. Calls `paymentRepository.deleteUploadedFile()` to delete PDF from folder
3. Then deletes record from `payments.txt`

How Feedback Works

Student Adding Feedback:

1. Student fills feedback form with `courseId`, rating 1 to 5, and comment
2. Calls `feedbackService.addFeedback()`
3. Service checks student has not already given feedback for same course
4. Service checks rating is between 1 and 5
5. Service checks comment is not empty
6. Creates Feedback with `submitted=false` and saves to `feedbacks.txt`

Student Editing Feedback:

1. Student submits edit form
2. Service checks feedback is not yet submitted
3. If not submitted — updates rating and comment in feedbacks.txt

Student Submitting Feedback:

1. Student clicks submit on feedback
2. Calls `feedbackService.submitFeedback()`
3. Service calls `feedback.submitFeedback()` which sets `submitted=true`
4. After this rating and comment cannot be changed
5. Repository updates feedbacks.txt with `submitted=true`

Lecturer Replying:

1. Lecturer types reply and submits
2. Calls `feedbackService.replyFeedback()`
3. Service checks reply is not empty
4. Calls `feedback.setReply()` and updates feedbacks.txt

Viewing by Role:

1. Controller reads `userRole` from session
2. If STUDENT — shows only that student's feedbacks
3. If LECTURER — gets all schedules for that lecturer, collects course IDs, shows feedbacks for those courses only
4. If ADMIN — shows all feedbacks

Average Rating:

1. `feedbackService.getAverageRating(courseId)` called
2. Gets all feedbacks for that course
3. Adds all ratings and divides by count
4. Returns average as double

How Schedule Works

Adding Schedule:

1. Admin fills schedule form with course, lecturer, day, start time, end time, room
2. Controller generates `scheduleId` using `"SCH" + System.currentTimeMillis()`
3. Gets Course, Lecturer, Room objects from their services
4. Calls `scheduleService.addSchedule()`
5. Service checks room is available
6. Service checks room capacity is enough for course max students
7. Service checks for time conflicts using `schedule.hasConflict()` against all existing schedules
8. If no conflicts — saves to schedules.txt

Viewing by Role:

1. If STUDENT — gets all approved registrations for that student, finds schedules matching those course IDs only
 2. If LECTURER — gets schedules where lecturerId matches
 3. If ADMIN — sees all schedules
-

How Room Management Works

Adding Room:

1. Admin fills room form with section, room number, capacity, availability
 2. Controller combines section and number to create roomId like A101
 3. Calls `roomService.addRoom()`
 4. Service checks roomId not already exists
 5. Service checks capacity is greater than zero
 6. Repository saves to `rooms.txt`
-

How Course Material Works

Lecturer or Admin Uploading:

1. Fills form with title, course, and uploads file
2. Controller checks role is LECTURER or ADMIN
3. Auto generates materialId using "MAT" + `System.currentTimeMillis()`
4. Auto detects file type from file extension
5. Calls `courseMaterialService.addCourseMaterial()`
6. Service checks file is not empty
7. Service checks file type is valid — PDF, PPTX, DOC, DOCX, PNG, JPG
8. Saves file to `uploads/materials/` folder with timestamp prefix
9. Saves record to `courseMaterials.txt`

Student Viewing:

1. Controller gets student's approved registrations
2. Collects all approved course IDs
3. Filters materials showing only those belonging to approved courses
4. Student cannot see materials for courses they are not approved in

Downloading:

1. `@GetMapping("/materials/download/{materialId}")` runs
2. Finds material gets filename
3. Builds path to `uploads/materials/` folder
4. Detects content type

5. PDF and images open directly in browser
 6. Other files like DOCX and PPTX are downloaded
-

How AuthInterceptor Works

1. Every single request passes through AuthInterceptor before reaching any controller
2. It checks if session has userRole attribute
3. If no session or no userRole — redirects to login page immediately
4. Controller never runs
5. Only login page, logout, and static files like CSS and JS are excluded from this check